

LUCA: A Distributed Banking System with Orchestrated Saga and File-Based Persistence

Boza Portilla, Yordano Hernan
Huacani Jara, Denise Andrea
Mamani Huarsaya, Jorge Luis
Mollo Chuquicaña, Dolly Yadhira
Pacheco Palo, Fabiana Francinet

Universidad Nacional de San Agustín de Arequipa, Escuela Profesional de Ingeniería de Sistemas, Sistemas Distribuidos 2026

Abstract — This paper presents LUCA, a distributed banking system that implements inter-bank transfers using the Orchestrated Saga pattern with automatic compensation. The system comprises three autonomous bank nodes (A, B, C) persisting financial state in flat JSON files with atomic writes, a centralized coordinator service for distributed transaction orchestration, and an API Gateway that unifies client access. Local operations—deposits, withdrawals, and validations—are executed with pessimistic locking (ReentrantLock + FileLock), SHA-256 blockchain-chained journaling, and shadow-paging atomic persistence. The Coordinator Service implements a two-step Saga (DEBIT origin → CREDIT destination) with automatic rollback on failure, ensuring financial consistency without distributed locks. All components are containerized with Docker and exposed through a single React frontend.

Resumen — Este artículo presenta LUCA, un sistema bancario distribuido que implementa transferencias interbancarias mediante el patrón Saga Orquestado con compensación automática. El sistema está compuesto por tres nodos bancarios autónomos (A, B, C) que persisten el estado financiero en archivos planos JSON con escritura atómica, un servicio coordinador centralizado para la orquestación de transacciones distribuidas y un API Gateway que unifica el acceso del cliente. Las operaciones locales—depósitos, retiros y validaciones—se ejecutan con bloqueo pesimista (ReentrantLock + FileLock), journal encadenado con SHA-256 y persistencia atómica con shadow-paging. El Coordinator Service implementa una Saga de dos pasos (DEBIT origen → CREDIT destino) con rollback automático ante fallos, garantizando la consistencia financiera sin bloqueos distribuidos. Todos los componentes se contenerizan con Docker y se exponen a través de un frontend único en React.

Keywords: microservices, API Gateway, Saga Pattern, Spring Boot, distributed transactions, file-based persistence, atomic writes, blockchain journaling, REST

1. Introduction

The modern financial landscape demands interoperability between banking entities operating in different regions. The LUCA project addresses this challenge by implementing a Distributed Banking Ecosystem composed of three autonomous institutions (Bank A, Bank B, Bank C) that operate under a technical federation model. Clients with presence in multiple banks can perform deposits, withdrawals, and inter-bank transfers transparently, without concern for the underlying distributed architecture.

The technical challenge lies not in moving funds between accounts, but in guaranteeing financial integrity in an environment without a centralized database. Each bank is the exclusive custodian of its data, stored locally in JSON files, introducing critical risks of consistency, concurrency, and atomicity. This paper describes how these challenges were addressed through the Orchestrated Saga pattern, pessimistic locking, atomic file writes, and immutable journaling.

2. System Requirements

2.1 Functional Requirements

The system must fulfill nine functional requirements (RF) organized into three domains:

User and Account Management:

- RF-01: Cross-border authentication and session management across all three banks.
- RF-02: Consolidated financial position view, aggregating accounts from all banks in parallel.
- RF-03: Transaction history query from both local bank files and the coordinator audit log.

Transaction Management and Interoperability:

- RF-04: Processing of local transfers (same bank) and inter-bank transfers (different banks).
- RF-05: Multi-operation chaining from a single client order.
- RF-06: Sufficient funds validation before initiating any lock or transfer phase.
- RF-07: Automatic reversal upon failure, executing a compensatory action to return funds to the origin account.

Storage and File Management:

- RF-08: Creation, reading, and updating of balances by directly modifying each bank's local flat files (JSON/text).
- RF-09: Temporary lock file generation (.tmp) during distributed transactions.

2.2 Non-Functional Requirements

Nine non-functional requirements (RNF) govern the system's quality attributes:

Consistency, Concurrency, and Synchronization:

- RNF-01: Exclusive OS-level file locking (FileLock) to prevent concurrent transaction corruption.
- RNF-02: Advanced coordination protocol with per-phase timeouts to avoid perpetual blocking.

- RNF-03: Global state persistence in a durable workflow store (Temporal).
- RNF-04: Controlled eventual consistency with balance updates within 3 seconds.

Architecture, Deployment, and Performance:

- RNF-05: Docker container isolation for each bank, the frontend, and infrastructure.
- RNF-06: REST/HTTP communication between Frontend-Backend and between microservices.
- RNF-07: Configurable data paths via environment variables.

Security and Usability:

- RNF-08: Immutable transaction auditing serving as a failure log.
- RNF-09: Real-time transactional flow status display in the React interface.

Frontend (React) | v API Gateway (port 8080) || Coordinator Svc (8090) | +- Bank A (8081) | +- Bank B (8082) | +- Bank C (8083) || Bank

3. Technology Selection

The technology stack was chosen to enable modular development and intercommunication among the three banking entities.

Architecture: Microservices. Each banking entity is represented as an independent service, facilitating separation among Bank A, Bank B, Bank C, and the Coordinator Service. This approach ensures isolated failure domains and independent scalability.

Backend: Spring Boot. Used for implementing banking entities and the coordinator service. Spring Boot enables ordered REST API creation, separates logic into controllers, services, and models, and facilitates independent application construction.

Frontend: React. Implements the web interface where users consult accounts, view balances, and request operations. React enables dynamic interfaces through reusable components and HTTP-based backend connectivity.

Communication: REST. Selected for the user plane due to its universality, browser compatibility, and ease of debugging with Postman. REST enables the coordinator to query and send operations to the corresponding banks.

Deployment: Docker and Docker Compose. Docker enables independent execution of Bank A, Bank B, Bank C, the Coordinator Service, and the frontend, simulating a truly distributed environment. Docker Compose orchestrates all services from a single docker-compose.yml.

Storage: JSON and LOG. JSON files store client and account information per bank (RNF-08). LOG files implement append-only transaction registries with blockchain-chained SHA-256 checksums for immutability (RNF-08).

Testing: Postman. REST endpoints are validated for account queries, deposits, withdrawals, and transfers.

Source Control: GitHub. Central repository for distributed development.

4. Proposed Architecture

4.1 General Architecture Description

The system is based on a Microservices Architecture with Distributed Transaction Orchestration. This model reflects the operational reality of an international banking consortium, where each banking node maintains total autonomy over its compute and persistence resources while interoperating under a common consensus protocol.

The architecture provides scalability and fault isolation: a critical failure in Bank A's file system does not compromise Bank B's or Bank C's operation, reducing the blast radius of technical errors.

4.2 Communication Flow

Figure 1: System communication flow through the API Gateway

4.3 API Gateway

The API Gateway (api-gateway-service, port 8080) serves as the system's front door. All external requests pass through it before reaching the coordinator or individual banks. It exposes unified endpoints: GET /api/customers/{id}/accounts, POST /api/operations/deposit, POST /api/operations/withdraw, POST /api/transfers, and GET /api/transactions/{id}.

Bank identification is resolved via account ID prefix: A- → Bank A, B- → Bank B, C- → Bank C. For local operations, the gateway acts as an intelligent router: it receives the frontend request, extracts the bank from the account prefix, and forwards the operation. For distributed transfers involving potentially different banks, the gateway transforms the frontend payload into a TransferRequest object compatible with the coordinator.

4.4 Coordinator Service (Saga Orchestrator)

The Coordinator Service implements the Orchestrated Saga pattern as the coordination mechanism for distributed transactions. This pattern divides each inter-bank operation into sequential local steps with compensatory transactions (logical rollbacks) upon any node failure. Two-Phase Commit (2PC) was discarded due to its vulnerability to coordinator failures (indefinite blocking), violating RNF-02's per-phase timeout requirement.

TransferController → SagaOrchestrationService | SagaTransaction + SagaStep + BankServiceClient SagaTransactionStore +

Figure 2: Saga Orchestrator internal architecture

The saga executes two sequential steps: (1) DEBIT on the origin account, (2) CREDIT on the destination account. If Step 2 fails, the executeCompensation() method analyzes the history of successful steps and issues a reverse CREDIT order on the origin account that was previously debited, preserving consistency and leaving the transaction in ABORTED state.

5. Local Operations Implementation

5.1 Domain Models

Each bank defines its own domain models to ensure separation of responsibilities. The Account entity encapsulates `accountId`, `clientId`, `bankCode`, `type`, `currency`, and `balance` (BigDecimal with scale 2). The Transaction model holds `transactionId` (UUID), `accountId`, `type` (DEBIT/CREDIT), `amount`, `balanceBefore`, `balanceAfter`, `timestamp`, and `status` (SUCCESS/FAILED/PENDING).

5.2 AccountController (REST Layer)

The AccountController serves as the direct contact point between the Coordinator Service and the bank's local file database. It exposes three endpoints:

Endpoint	Method	Function
/clients/{clientId}/accounts	GET	List client accounts
/api/v1/bank/accounts/{id}/debit	POST	Saga debit
/api/v1/bank/accounts/{id}/credit	POST	Saga credit

Table 1: AccountController endpoints

The controller uses a functional interface `AccountOperation` and a template method `applyOperation()` that unifies error handling: `INVALID_AMOUNT` → 400, `InvalidAmountException` → 400, `InsufficientFundsException` → 400, `IllegalArgumentException` → 400, `IllegalStateException` → 409. The inner class `BankOperationRequest` serves as the input DTO.

```
@PostMapping("/api/v1/bank/accounts/{id}/debit")
public Map<String, Object> debit(
    @PathVariable String id,
    @RequestBody BankOperationRequest req) {
    String txId = resolveTransactionId(req);
    return applyOperation(
        () -> service.debit(txId, id, req.getAmount()),
        id, txId, req);
}
```

5.3 AccountOperationService (Core Business Logic)

The AccountOperationService implements two differentiated execution patterns:

Local operations (deposit/withdraw): Full lifecycle with pre-validation outside the lock, re-validation inside the lock, journaling, atomic write, and return of Transaction with status. The withdraw operation implements an optimistic pre-check pattern: it validates the balance before acquiring the lock, but re-validates under lock to protect against race conditions.

Saga operations (debit/credit): Lightweight versions designed to be invoked by the orchestrator. They do not validate funds—that responsibility falls on the Coordinator. The entire

logic occurs within the lock, and a `sagaId` is concatenated to the journal's `transactionId` for end-to-end traceability.

5.4 AccountValidationService

The validation service centralizes all pre-execution rules and is used exclusively by `deposit()` and `withdraw()`. Methods include: `validateDeposit(account, amount)` — account not null, amount > 0; `validateWithdraw(account, amount)` — account not null, amount > 0, balance >= amount; `calculateNewBalance(account, type, amount)` — DEPOSIT adds, WITHDRAW subtracts, scale 2 with HALF_EVEN rounding.

5.5 Concurrency and Atomicity (common-persistence)

The shared common-persistence library provides three critical infrastructure services:

NioFileLockManager: Implements two-tier exclusive locking: (1) JVM-level `ReentrantLock.tryLock(timeout)` serializes threads within the same process; (2) OS-level `FileChannel.lock()` protects against external processes. The `acquireExclusiveLock()` method returns an `AutoCloseable` that releases both locks.

JsonAtomicFileWriter: Implements shadow-paging: writes JSON to `accounts.json.tmp` with `channel.force(true)` (fsync), then performs `Files.move(tmp, target, ATOMIC_MOVE)` — the OS-level atomic rename guarantees the file is never left in a corrupt intermediate state.

FileJournalingService: Implements an immutable blockchain-chained journal with SHA-256. Each entry contains `previousHash` (the hash of the prior entry, null for genesis) and `checksum` = SHA-256(`txId` + `accountId` + `operationType` + `amount` + `prevBalance` + `newBalance` + `previousHash`). Any alteration to an entry invalidates all subsequent checksums.

5.6 Replication Across Banks

All three banks share an identical 12-file structure, differing only in port, package, bank code, and data files:

Bank	Port	Package	Code	Accounts File
A	8081	pe.unsa.sd.bank	BANK_A	accounts_A.json
B	8082	pe.unsa.sd.bank	BANK_B	accounts_B.json
C	8083	pe.unsa.sd.bank	BANK_C	accounts_C.json

Table 2: Configuration per bank

6. Distributed Transfers with Orchestrated Saga

6.1 Saga Transaction Model

The root entity is `SagaTransaction`, representing the totality of a distributed transaction. It contains a list of `SagaStep` entries, each encapsulating an atomic operation (DEBIT or CREDIT) with its own state (PENDING, EXECUTING, SUCCESS, FAILED, COMPENSATED).

SagaStatus	Description
PENDING	Transaction created, awaiting execution
EXECUTING_STEP_1	Debit on origin bank in progress
EXECUTING_STEP_2	Credit on destination bank in progress
COMMITTED	Both steps succeeded
COMPENSATING	Rollback in progress
ABORTED	Compensation completed, funds restored

Table 3: Saga states and transitions

6.2 Execution Flow

Coordinator → Step 1: DEBIT in origin bank 4. Coordinator → Step 2: CREDIT in dest. bank 5a. Both OK → COMMITTED, completedAt set 5b. Step 2 fails → COMPENSATING → CREDIT origin → ABORTED

Figure 3: Saga execution flow

6.3 Automatic Compensation

In strict compliance with RF-07, if the destination bank throws an exception (non-existent account or timeout), the catch block invokes `executeCompensation()`. This method analyzes successful step history and issues a reverse order (CREDIT on the previously debited origin account), preserving consistency and leaving the transaction in ABORTED state. The response includes the complete step history with timestamps and error messages.

6.4 Integration with Bank Nodes

Each bank exposes `POST /api/v1/bank/accounts/{id}/debit` and `POST /api/v1/bank/accounts/{id}/credit` endpoints that the Coordinator calls via `BankServiceClient` (RestTemplate). The orchestrator depends exclusively on HTTP status codes to decide whether to advance to the next saga step or abort and initiate compensation.

7. Results

7.1 Endpoint Validation

All endpoints were validated using Postman. The `AccountController` endpoints respond with semantically correct HTTP codes: 200 OK on success, 400 Bad Request on invalid amounts or insufficient funds, and 409 Conflict on lock failures. The `POST /api/v1/orchestrator/transfers` endpoint returns 200 OK for COMMITTED transactions and 409 Conflict for ABORTED transactions with complete step history.

7.2 Local Operations

Deposit operations correctly increase balance and register journal entries. Withdraw operations decrease balance and validate sufficient funds before execution. Debit and credit operations for Saga execute without fund validation (coordinator responsibility), returning the new balance as `BigDecimal`.

7.3 Distributed Transfers

The saga successfully executes the DEBIT → CREDIT sequence between different banks. When the destination account is invalid or unreachable, the automatic compensation mechanism reverses the origin DEBIT, restoring the original balance. The transaction log preserves the complete trace for auditability.

7.4 Concurrency Control

The two-tier locking mechanism serialized all concurrent requests without `OverlappingFileLockException` errors. The `ReentrantLock` prevents JVM-level thread overlap, while `FileChannel.lock()` provides OS-level protection against external processes.

8. Conclusions

This project demonstrates the feasibility of implementing a fully functional distributed banking system using flat file persistence and the Orchestrated Saga pattern. The developed solution achieves financial consistency across three autonomous bank nodes through a combination of pessimistic locking, atomic shadow-paging writes, blockchain-chained journaling, and automatic saga compensation.

The API Gateway provides location transparency, enabling clients to operate without knowledge of the internal microservice topology. The Coordinator Service guarantees that distributed transactions are either fully committed or fully compensated, leaving the system in a consistent state after any failure. Local operations (deposit, withdraw) maintain account integrity through rigorous pre- and post-lock validation, while saga operations (debit, credit) are deliberately lightweight to keep the orchestrator's concerns separated from the bank nodes.

The architecture supports horizontal scalability: adding a new bank requires copying the structure, changing the package, port, and data files—no code coupling exists between nodes. The immutable journal with SHA-256 chaining provides a tamper-evident audit trail that meets the strictest financial audit requirements.

Future work includes integration with Temporal for durable workflow execution (RNF-03), implementation of the full Three-Phase Commit protocol with timeouts (RNF-02), and addition of containerized deployment with Docker Compose for reproducible environment setup.

REFERENCES

1. C. Richardson, *Microservices Patterns*. Shelter Island, NY: Manning Publications, 2019.
2. H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Francisco, CA, 1987, pp. 249--259.
3. P. A. Bernstein and N. Goodman, "Concurrency control in distributed database systems," *ACM Comput. Surv.*, vol. 13, no. 2, pp. 185--221, Jun. 1981.
4. Spring Boot Documentation, "Spring Boot Reference Guide," 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

5. Docker Inc., "Docker Compose Overview," 2024. [Online]. Available: <https://docs.docker.com/compose/>
6. P. Bailis and A. Ghodsi, "Eventual consistency today: Limitations, extensions, and beyond," *_Commun. ACM_*, vol. 56, no. 5, pp. 55--63, May 2013.
7. G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *_Distributed Systems: Concepts and Design_*, 5th ed. Boston, MA: Addison-Wesley, 2012.
8. React Documentation, "React: A JavaScript library for building user interfaces," 2024. [Online]. Available: <https://react.dev/>